Fine-Tuning the VGGT Depth Head on Driving Scenes

FatemehZahra Ghafari Ghomi

Project Work in Image Processing and Computer Vision

Supervisor: Prof. Giuseppe Lisanti

University Of Bologna

October 2026

Contents

1 Introduction **2**

2 Background: The 3D Reconstruction Problem **3**
2.1 What We Want to Recover . . . 3
2.2 How the Problem Was Solved Before . . . 3
2.3 What VGGT Proposes . . . 4
2.4 Driving Scenes in VGGT's Training Data . . . 4

3 The VGGT Architecture **5**
3.1 From Images to Tokens . . . 5
3.2 The Alternating-Attention Transformer . . . 5
3.3 Prediction Heads . . . 6
3.4 Training . . . 6
3.5 What This Means for Our Project . . . 7

4 Replication on an Example Scene **8**

5 Datasets **10**
5.1 Virtual KITTI 2 . . . 10
5.2 Real KITTI . . . 10

6 Evaluation Protocol **11**

7 Experiments and Results **12**
7.1 Compute Environment . . . 13
7.2 Fine-Tuning Configuration . . . 13
7.3 Training and Epoch Selection . . . 13
7.4 Lessons from a First Attempt . . . 14
7.5 Baseline: Pretrained VGGT . . . 14
7.6 Experiment 1: Full Resolution . . . 15
7.6.1 Training . . . 15
7.6.2 Epoch Selection . . . 16
7.6.3 Test Results . . . 17
7.6.4 View-Count Ablation . . . 18
7.6.5 Further Tests . . . 18
7.7 Experiment 2: Lower Resolution . . . 19

8 Conclusion **21**

1 Introduction

A self-driving car or a robot has to understand the 3D world around it: the shape of the road, and how far away the other cars and buildings are. Special sensors such as LiDAR can measure distances directly, but they are expensive, while cameras are cheap and available almost everywhere. A photo, however, is only a flat 2D image, so the 3D information has to be recovered from it. This is a classic problem in computer vision: given a few photos of the same place, we want to find where each photo was taken from, how far each pixel is from the camera, and where each point of the scene is in 3D.

For many years, this was done with geometric methods that match points between the photos and then run a long, iterative optimization to find the cameras and the 3D points. These methods can be accurate, but they are slow and they fail easily when there are only a few photos. VGGT (Visual Geometry Grounded Transformer) (Wang et al., 2025) takes a different approach. It is a large transformer that looks at all the photos at once and directly predicts the camera parameters, depth maps, point maps, and point tracks, in a single forward pass and without any optimization step.

VGGT was trained on 17 datasets, but only a few of them contain street or driving scenes, and none of them is a real driving dataset with LiDAR depth like KITTI. This matters, because driving scenes are different from most other scenes: the images are very wide, the camera usually moves forward, and objects can be very close or very far away. So it is not clear how well VGGT works on real roads, and whether a small adaptation could make it better.

To study this, we fine-tune VGGT on Virtual KITTI 2 (Cabon et al., 2020), a synthetic driving dataset in which the depth of every pixel is known exactly. Virtual KITTI is already one of VGGT's training datasets, so testing on it would only show how well the model fits data it has already seen. For this reason, we mainly test the model on the real KITTI dataset (Geiger et al., 2013), which VGGT has never seen. In other words, we train on synthetic images and test on real ones.

All experiments ran on a single Google Colab GPU, so we had to use a much lighter training setup than the original one. We froze the large transformer backbone and trained only the depth head. We also used fewer views of the same scene in each training sample (2 to 8 instead of up to 24), and in a separate experiment we reduced the image resolution as well.

With this setup, we try to answer four questions:

- Does fine-tuning only the depth head on synthetic driving data improve the depth on synthetic driving scenes?
- Does this improvement also hold on real driving images that the model has never seen?

- How does the number of input views (2, 4, 8, or 16) change the depth quality on real driving scenes?
- How much does a lower image resolution reduce the depth quality, and can fine-tuning the depth head make up for it?

The code is available on GitHub[1].

2 Background: The 3D Reconstruction Problem

2.1 What We Want to Recover

Imagine we have a few photos of the same street, taken from slightly different positions. We call them $(I_i)_{i=1}^N$. From these photos alone, we want to recover three things for each image I_i:

- **the camera parameters** g_i: where the camera was, which way it was pointing, and how wide its view was;
- **the depth map** $D_i \in \mathbb{R}^{H \times W}$: for each pixel, how far the scene point is from the camera, measured along the camera's viewing direction;
- **the point map** $P_i \in \mathbb{R}^{3 \times H \times W}$: for each pixel, the 3D coordinates (x, y, z) of the scene point.

To give 3D coordinates, we first need a reference point. Since the photos do not tell us where the "world" is, VGGT simply uses the **first camera** as the reference: the first camera is at the origin, and all other cameras and points are described relative to it. VGGT can also follow the same point across the images (point tracks), but we do not use this in our project.

The three outputs are closely connected. If we know the depth map of an image and where its camera was, we can compute the point map directly, by projecting each pixel back into 3D. So the depth map is one of the most useful outputs, and it is the one we focus on in this project.

2.2 How the Problem Was Solved Before

For a long time, the standard solution was **Structure-from-Motion (SfM)** (Hartley and Zisserman, 2004), with COLMAP (Schönberger and Frahm, 2016) as the best-known tool. SfM looks for the same points in different photos, computes where they are in 3D, and then runs **Bundle Adjustment**, a large optimization that slowly adjusts all cameras and 3D points until they fit the photos as well as possible. The results can be very accurate, but the process is slow, and it often fails when there are only a few photos or when the photos do not overlap much.

[1] https://github.com/arisaghafari/vggt-depth-finetune

Deep learning later made parts of this pipeline better. VGGSfM (Wang et al., 2024a) made the whole pipeline trainable, but it still needs Bundle Adjustment at the end. DUSt3R (Wang et al., 2024b) and MASt3R (Leroy et al., 2024) went one step further, because their network directly outputs 3D points from the photos. The problem is that they can only look at **two photos at a time**. For a whole scene, they have to combine the results of many pairs, and this again needs a slow optimization step. So, even with deep learning, the slow optimization never really went away.

2.3 What VGGT Proposes

VGGT gets rid of the optimization completely. It is a single neural network that looks at all the photos of a scene together and directly outputs everything from Section 2.1 in one forward pass. It does not work on pairs: it can take one photo, a few photos, or even hundreds of them at once. It also does not need a separate model for each task, because one shared network predicts the cameras, depth maps, point maps, and tracks together. What is perhaps most surprising is how simple the network is. It is a fairly standard large transformer, with almost nothing designed specifically for 3D. Just like large language models learn language from huge amounts of text, VGGT learns geometry from a large amount of labelled 3D data instead of hand-written geometric rules.

This simple idea works very well. According to the paper, VGGT estimates the cameras of 10 photos in about 0.2 seconds, while DUSt3R, MASt3R, and VGGSfM need 7 to 10 seconds and are still less accurate (Wang et al., 2025).

2.4 Driving Scenes in VGGT's Training Data

VGGT was trained on 17 datasets, and only three of them show streets or driving scenes. Two of them, Virtual KITTI and MVS-Synth, are synthetic, and the third one, Mapillary, contains real street-level photos. This means that Virtual KITTI, the dataset we used for fine-tuning, is not new to the model.

We could have used a driving dataset that VGGT has never seen, but in practice this was much harder. VGGT's official training code already has a data loader for Virtual KITTI, while any other dataset would have needed a new one. This loader has to provide the camera position for every frame, and real driving datasets usually give this only indirectly, through GPS and other sensor data. The training loss also needs the depth of every pixel. A synthetic dataset gives this for free, but real datasets only have sparse LiDAR points. On top of that, most large driving datasets are hundreds of gigabytes, which is difficult to handle on Google Colab.

Virtual KITTI was also a good match for what we wanted to study. It is a copy of real KITTI scenes, rebuilt in a game engine, so both datasets show similar roads and differ mainly in how the images look: rendered or real. So, by fine-tuning on Virtual KITTI

and testing on real KITTI, which VGGT has never seen, we could check whether what the model learns from synthetic images also helps on real photos.

3 The VGGT Architecture

VGGT turns a set of images into the outputs of Section 2.1 in three steps (Figure 1). First, each image is cut into small pieces and turned into a sequence of tokens. Then, a large transformer, called the *aggregator*, lets the tokens of all images exchange information. Finally, small prediction heads turn the tokens into cameras, depth maps, point maps, and tracks. Together, these parts have about 1.2 billion parameters, and almost all of them are in the aggregator.

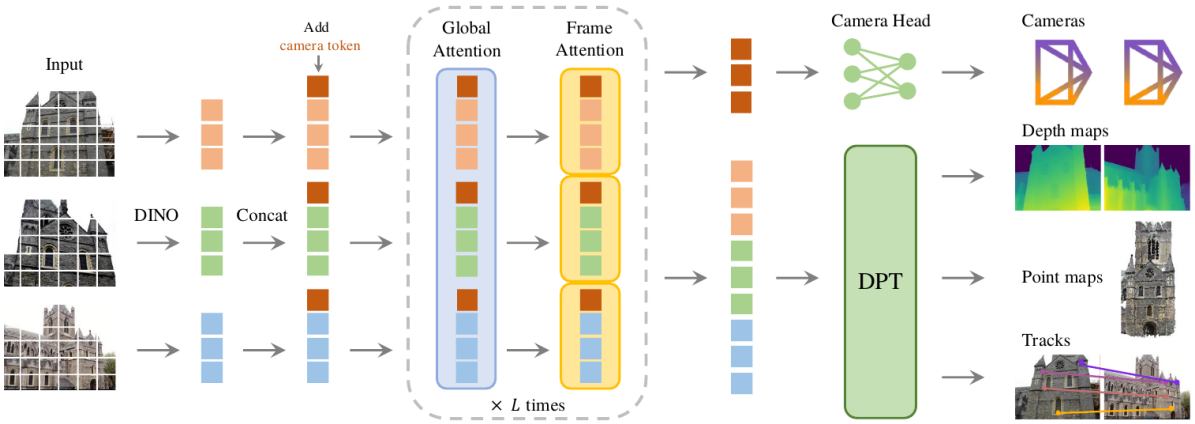


Figure 1: Architecture overview, reproduced from Wang et al. (2025). Each image is split into tokens by DINO, a camera token is added to each frame, and the tokens pass through L blocks that alternate between global and frame-wise attention. A camera head predicts the cameras, and a DPT head predicts the dense outputs.

3.1 From Images to Tokens

A transformer cannot work on an image directly. It needs a sequence of vectors, called tokens. VGGT creates them with a pretrained **DINOv2** model (Oquab et al., 2024): each image is cut into patches of 14×14 pixels, and each patch becomes one token. This means that the number of tokens depends on the image size, so a smaller image gives fewer tokens and is cheaper to process. We come back to this in our low-resolution experiment (Section 7.7). Each image also gets an extra **camera token**, which collects the information needed to predict its camera. The first image gets a different camera token from all the others, and this is how the model knows which image is the reference.

3.2 The Alternating-Attention Transformer

The aggregator is the heart of VGGT. It is a standard transformer with 24 blocks, but with one change, which the authors call **Alternating-Attention**. Each block has two

attention layers that look at the tokens in two different ways. In the **frame-wise** layer, each token only looks at the other tokens of its own image, which helps the model understand each image on its own. In the **global** layer, the tokens of all images look at each other, which lets the model find the same points in different images and reason about the 3D shape of the scene. The global layers are the reason why VGGT can benefit from more views: the more images it sees together, the more information it has about the scene. The authors show that this alternating design works better than using only global attention or using cross-attention between images.

3.3 Prediction Heads

When the tokens leave the aggregator, they go to small prediction heads, and each head has a single job. Besides the depth head, VGGT has heads for the cameras, the point maps, and the point tracks, but in this project we only use the depth head.

The depth head has to solve a practical problem: the aggregator works on patches of 14×14 pixels, but depth is needed for every single pixel. VGGT solves this with a **DPT** decoder (Ranftl et al., 2021), which is designed for this kind of dense prediction with transformers. It takes tokens from four different blocks of the aggregator. The early blocks contain fine details such as edges, while the later ones contain a more global view of the scene. The decoder combines them and upsamples them step by step, back to the full image size. For each pixel, it then outputs the depth $\hat{D}_i$ and a **confidence** value Σ_i^D, which says how sure the model is about that pixel.

The paper also contains a result that supports our choice of the depth head. Since the outputs are related, the 3D points can be computed in two ways: read directly from the point map head, or computed from the predicted depth and the predicted camera. The second way gives more accurate points (an Overall error of 0.677 instead of 0.709 on ETH3D). Splitting the hard problem into two easier ones, depth and camera, works better, and this makes the depth head one of the most useful parts of the model.

3.4 Training

Depth loss. VGGT is trained with one loss for each task, and the losses are added together. For us, only the **depth loss** matters, because it is the loss we used for fine-tuning:

$$\mathcal{L}_{\text{depth}} = \sum_{i=1}^{N} \left\| \Sigma_i^D \odot (\hat{D}_i - D_i) \right\| + \left\| \Sigma_i^D \odot (\nabla \hat{D}_i - \nabla D_i) \right\| - \alpha \log \Sigma_i^D$$

Each of its three terms has a clear role. The first term is the depth error at each pixel, weighted by the confidence Σ_i^D. The second term compares the *gradients* of the depth maps, that is, how the depth changes from one pixel to the next. It pushes the model to

produce sharp object edges instead of blurry depth. The third term, $-\alpha \log \Sigma_i^D$, stops the model from cheating. Without it, the model could set its confidence to zero everywhere and make the first two terms disappear. With it, low confidence has a cost, so the model learns to be confident where it is accurate and less confident on difficult pixels, such as the sky or object edges. This idea is known as an aleatoric-uncertainty loss (Kendall and Gal, 2017).

Scale of the ground truth. From photos alone, it is impossible to know the real size of a scene. A photo of a real car taken from 10 metres away can look exactly the same as a photo of a toy car, 100 times smaller, taken from 10 centimetres away. Photos can tell us the *shape* of a scene, that is, which points are closer and which are farther, and by how much, but not whether the distances are in metres or centimetres. The training data comes from many different datasets, so the authors first bring every scene to the same scale. They express all 3D points in the coordinate system of the first camera, compute the average distance of these points from the camera, and divide all ground-truth values by this number. After this step, every scene, whether it is a small kitchen or a long street, has an average point distance of 1. The network learns to predict depth in this relative unit, not in metres. This means that the ratios between VGGT's depth values are meaningful, for example that one car is twice as far away as another, but the values themselves are not real distances. Before comparing VGGT's depth with the KITTI ground truth in metres, we therefore have to rescale it, as described in Section 6.

Training setup. The original model was trained for 160K iterations with the AdamW optimizer. Each batch has 48 frames, taken from random scenes in groups of 2 to 24 frames. The images are resized so that their longest side is 518 pixels, with a random aspect ratio between 0.33 and 1.0. Training used bfloat16 precision and took nine days on 64 A100 GPUs.

3.5 What This Means for Our Project

Nine days on 64 A100 GPUs is far beyond what we had, which was a single A100 GPU on Google Colab. Fortunately, the architecture allows a much lighter adaptation. The aggregator holds almost all of the parameters and does all of the reasoning across images, while the depth head only turns its tokens into depth. So we froze the aggregator and trained only the depth head, with the depth loss above. In this way, the model keeps the general knowledge about 3D geometry that it learned from 17 datasets, and only the last step, turning this knowledge into depth, is adapted to driving scenes.

Some details of the original training also had to change for our experiments. The original setup used up to 24 frames per sample and 48 frames per batch, which needs much more memory and time than a single GPU can offer, so we used far fewer frames. It also used

images with a longest side of 518 pixels, and in a separate experiment we tested a lower resolution to reduce the cost even further. Finally, its aspect ratios (0.33 to 1.0) do not fully cover KITTI images, which are about 3.3 times wider than they are tall. Section 7 explains how we handled each of these.

4 Replication on an Example Scene

Before working with driving data, we wanted to be sure that VGGT ran correctly in our setup. So we first tried the official pretrained VGGT-1B model on one of the example scenes that come with the official code, called `kitchen`. It contains 25 photos of a LEGO bulldozer on a table, taken from different positions around it. We gave all 25 photos to the model together, in a single forward pass, and used three of its heads: the camera head, the depth head, and the point map head.

The model produced all the expected outputs for every photo: the camera parameters, a depth map with its confidence map, and a point map. The outputs have a size of 350×518 pixels, because VGGT resizes each photo to a width of 518 pixels and keeps its shape, rounding the height to a multiple of the 14-pixel patch size. Since this scene has no ground truth, we could only check the results visually.

The depth map looks as expected (Figure 2). The bulldozer is clearly separated from the table, and the table becomes gradually farther away towards the back of the scene. The confidence is high on the table and on the bulldozer, and lower in the background, such as the window and the plant, and along the edges of the bulldozer, where depth is hardest to estimate.

We then combined the predicted depth with the predicted cameras to turn every pixel into a 3D point, which is the method the paper recommends (Section 3.3). Seen from above (Figure 3), the reconstruction shows the table, the two mats, and the bulldozer in the right positions. The 25 estimated cameras surround the scene, as expected for photos taken around a table, and the first camera lies at the origin, because it is the reference for all the other results.

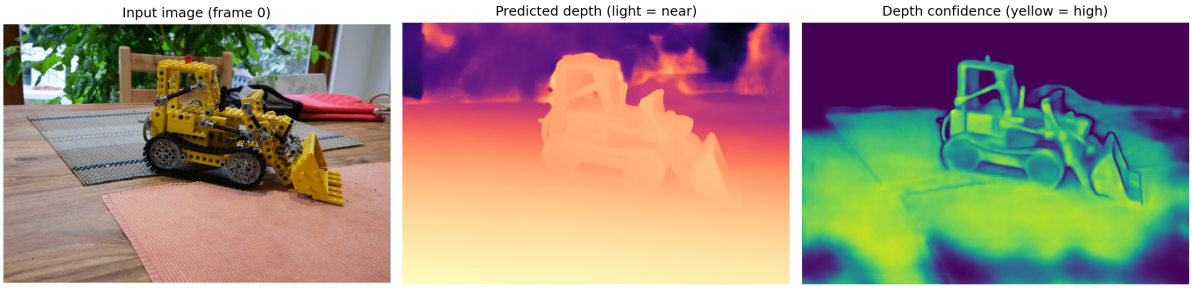


Figure 2: Replication on the `kitchen` example scene, first photo: input image (left), predicted depth, where lighter means closer (middle), and depth confidence, where yellow means more confident (right).

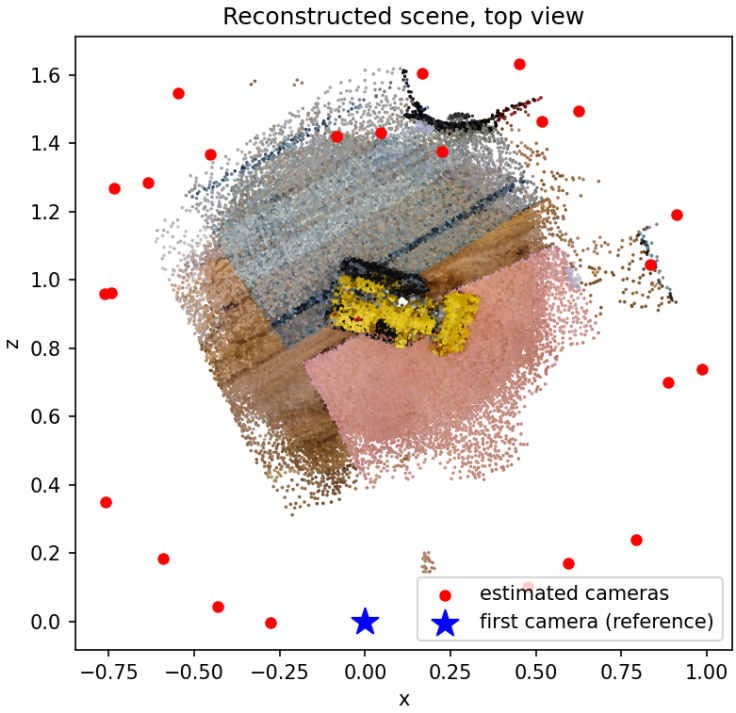


Figure 3: Top view of the reconstructed `kitchen` scene: 3D points computed from the predicted depth and cameras (coloured by the photos), the 25 estimated camera positions (red), and the first camera, which is the reference (blue).

These results showed that the pretrained model, the preprocessing, and the heads all worked as expected, so we used the same setup for the experiments on driving data.

5 Datasets

As explained in Section 2.4, we fine-tuned on Virtual KITTI 2 and tested mainly on real KITTI, although our original plan was to fine-tune directly on real KITTI. This section describes how we used the two datasets.

5.1 Virtual KITTI 2

In Virtual KITTI 2 (Cabon et al., 2020), each scene is a short drive through a virtual town, rebuilt from a real KITTI recording, with images of 1242×375 pixels. Since the images are rendered, the depth of every single pixel is known exactly, and the camera parameters of every frame are also given. We used the default version of each scene and the left camera, `Camera_0`.

The dataset has only five scenes in total, so we used all of them: four for training and Scene02, the shortest one, for testing (Table 1). Since the model has most likely seen Scene02 during its original training, this test only shows how much the depth head can adapt to data it already knows.

Scene	Frames	Use
Scene01	447	Training
Scene06	270	Training
Scene18	339	Training
Scene20	837	Training
Scene02	233	Test (160 frames evaluated)

Table 1: Use of the Virtual KITTI 2 scenes (default version, `Camera_0`).

5.2 Real KITTI

The KITTI dataset (Geiger et al., 2013) was recorded by a car driving around Karlsruhe, in Germany. Besides its cameras, the car has a Velodyne LiDAR scanner, which is calibrated with the cameras, so its 3D points can be projected into the images to give the true depth at those pixels. We used the ground truth of the KITTI depth benchmark (Uhrig et al., 2017), which combines several LiDAR scans to get denser depth maps. Even so, the ground truth is still **sparse**: there are no points in the sky, in the upper part of the image, or on some shiny or very distant surfaces.

To keep the download and the evaluation time small on Colab, we used three short drives from the validation part of the depth benchmark as **test drives**: `2011_09_26_drive_0002`, `_0005`, and `_0013`. From each drive, we took 64 frames with ground truth from the left colour camera (`image_02`), for a total of 192 frames. We also used three other drives from the same part of the benchmark as **validation drives**, only to choose the best training

epoch: `2011_09_26_drive_0020`, `_0036`, and `_0079`, again with 64 frames each. The validation and test drives never overlap.

6 Evaluation Protocol

To compare the pretrained and the fine-tuned models fairly, we evaluated all of them with exactly the same code. In this way, any difference in the results comes from the model and not from the evaluation.

Multi-view input. A KITTI drive is a video, so neighbouring frames show almost the same part of the street from slightly different positions. VGGT works best when it sees several such frames together, because it can compare them with each other. So we split the frames of each drive into small groups of consecutive frames, which we call windows, and gave each window to the model in one forward pass. Unless stated otherwise, each window has 8 frames, so the 64 frames of a drive form 8 windows. The windows do not overlap, so every frame belongs to exactly one window and is evaluated exactly once. Since 64 can be divided by 2, 4, 8, and 16, the same 64 frames are evaluated whatever the window size, and in the view-count experiment (Section 7.6.4) only the number of frames that the model sees together changes.

Feed-forward only. We always used VGGT in its feed-forward mode, without bundle adjustment, and took the depth maps directly from the depth head. In this way, we measured only what the depth head itself produces, which is the part we fine-tuned.

Resolution. We prepared the images with VGGT's standard function, which resizes them to a width of 518 pixels, so that a KITTI frame becomes 518×154 pixels. In the low-resolution experiment (Section 7.7), we used our own resizing function with a width of 336 pixels. In both cases, we resized the predicted depth map back to the size of the ground truth before comparing them.

Scale alignment. As explained in Section 3.4, VGGT does not know the real size of a scene, so its depth values are in a relative unit, while the ground truth is in metres. The two cannot be compared directly. Since VGGT gets the relations between pixels right, for example that one car is twice as far away as another, we only need one number to convert its depth into metres. For each frame, we take the median of the true depths and the median of the predicted depths over the valid pixels, and multiply the whole predicted depth map $\hat{D}$ by their ratio:

$$s = \frac{\text{median}(D)}{\text{median}(\hat{D})}.$$

After this step, the typical predicted depth matches the true one, and the errors that remain are real errors in the shape of the depth map. We use the median rather than

the average because a few extreme pixels cannot change it. This method, called *median scaling*, was introduced for depth models that do not know the real scale of a scene (Zhou et al., 2017), and it has become the standard way to evaluate them on KITTI. We apply it in exactly the same way to every model we compare.

Valid pixels. We evaluated only the pixels that have a ground-truth depth between 0.001 and 80 metres. For KITTI, we also used the standard crop of Garg et al. (2016), which keeps only the lower central part of the image, where the LiDAR has measurements.

Metrics. We report the standard depth metrics of Eigen et al. (2014). Here, d is the ground-truth depth of a valid pixel, $\hat{d}$ is the scaled prediction, and the averages are taken over all valid pixels of a frame:

- **AbsRel** $= \text{mean}\left(|\hat{d} - d|/d\right)$: the average error as a fraction of the true depth (lower is better);
- **SqRel** $= \text{mean}\left((\hat{d} - d)^2/d\right)$: like AbsRel, but large errors count more (lower is better);
- **RMSE** $= \sqrt{\text{mean}\left((\hat{d} - d)^2\right)}$: the typical error in metres (lower is better);
- **RMSE log** $= \sqrt{\text{mean}\left((\log \hat{d} - \log d)^2\right)}$: the same in log space, so that near and far points count more equally (lower is better);
- $\delta < 1.25^k$: the fraction of pixels where the prediction is within a factor 1.25^k of the truth, that is, $\max(\hat{d}/d,\ d/\hat{d}) < 1.25^k$, for $k = 1, 2, 3$ (higher is better).

We computed the metrics for each frame and then averaged them over all frames. We report all seven metrics for completeness, but in the discussion we focus on two of them: AbsRel, which gives the average error, and $\delta < 1.25$, which gives the share of pixels that are close enough to the truth. We use the other metrics only when they show something that these two do not.

Finally, the two datasets have very different ground truth: Virtual KITTI has depth for every pixel up to 80 metres, while KITTI has sparse LiDAR points, mostly on the road and on nearby objects. So the metric values should only be compared *within* one dataset, for example before and after fine-tuning, and not between the two datasets.

7 Experiments and Results

We ran two experiments with the same training and selection procedure. In the first one, we fine-tuned the depth head at VGGT's normal input width of 518 pixels. In the second one, we repeated it at a lower width of 336 pixels, to see how much a cheaper setup costs and whether fine-tuning can make up for it.

7.1 Compute Environment

We ran all experiments on Google Colab, on a single NVIDIA A100 GPU, both for fine-tuning and for evaluation. We used the official pretrained `VGGT-1B` weights and the official VGGT training code.

7.2 Fine-Tuning Configuration

In both experiments, we froze the aggregator and trained only the depth head, with the depth loss of Section 3.4. The camera, point map, and tracking heads were switched off. Table 2 lists the settings, and shows how much lighter they are than the original training of VGGT.

Setting	Value
Training data	Virtual KITTI 2, Scenes 01, 06, 18, 20
Image width	518 pixels (Experiment 1), 336 pixels (Experiment 2)
Aspect ratio (height/width)	randomly chosen between 0.28 and 0.34
Frames per sample	2 to 8 (original training: 2 to 24)
Images per GPU	at most 8 (original training: 48)
Optimizer	AdamW, weight decay 0.05
Learning rate	5×10^{-5}, 5% linear warm-up, then cosine decay
Gradient clipping	maximum norm 1.0
Training length	3,000 steps in 30 epochs (original training: 160K iterations)
Precision	bfloat16 mixed precision

Table 2: Fine-tuning settings shared by both experiments.

7.3 Training and Epoch Selection

Training longer on synthetic data does not always make the model better on real images. So, instead of simply taking the model after the last step, both experiments followed the same three steps.

Training. We trained the depth head for 3,000 steps, split into 30 epochs of 100 steps. In VGGT's training code, an epoch is a fixed number of steps, not a full pass over the training data. After every epoch, we saved a copy of the depth head.

Epoch selection with early stopping. We evaluated the pretrained depth head and the 30 saved ones on the three real-KITTI validation drives (Section 5.2). Going through the epochs in order, we kept the one with the lowest AbsRel so far, and stopped when it had not improved for five epochs in a row. The depth head kept at that point is the *selected* model. In this way, real labelled data was used only to choose an epoch, never to train, and the test drives played no role in the choice.

Testing. We evaluated two depth heads on the test sets: the selected one, and the one of the last epoch, which shows what happens if we simply train and take the result. Only after the choice, as a check, we also evaluated every epoch on the test drives, to see whether the validation drives rank the epochs in the same way.

7.4 Lessons from a First Attempt

Before these experiments, we made a first fine-tuning attempt, trained on Scene01 only for 3 epochs of 50 steps and evaluated on 15 frames of Scene02. The fine-tuned model was clearly worse than the pretrained one (AbsRel 0.062 against 0.039). Looking at this run more closely, we found three problems, which the settings above correct:

- **Too little training data.** One scene shows one virtual town, so the depth head could overfit to it; we now use four scenes.
- **Wrong aspect ratio.** Ratios between 0.3 and 3.5 gave images up to 3.5 times *taller* than wide, unlike KITTI's wide frames (about 0.30); we now use values around 0.30.
- **Different resolutions for training and testing.** The run was trained at width 224 but tested at 518; we now use the same width for both.

Because of these problems, we do not use the first attempt in any comparison.

7.5 Baseline: Pretrained VGGT

Table 3 shows the pretrained model on both test sets, with windows of 8 frames. On both datasets, it is already accurate: the average error is about 4 to 5% of the true depth, and about 97% of the pixels are within 25% of the truth.

Test set	Frames	AbsRel ↓	SqRel ↓	RMSE ↓	RMSE log ↓	δ_1 ↑	δ_2 ↑	δ_3 ↑
Virtual KITTI 2 (Scene02)	160	0.0384	0.4814	4.145	0.1055	0.9743	0.9851	0.9924
Real KITTI (3 drives)	192	0.0489	0.2327	2.932	0.0861	0.9695	0.9933	0.9974

Table 3: Pretrained VGGT with 8-frame windows and median scaling. δ_k denotes $\delta < 1.25^k$.

The average error is higher on real KITTI (AbsRel 0.049 against 0.038), as we expected for real images from a dataset the model has not seen. The RMSE is lower on KITTI, but this does not mean that KITTI is easier: Virtual KITTI has ground truth up to 80 metres, including distant buildings where errors in metres are large, while the KITTI LiDAR mostly covers the road and nearby objects.

Figure 4 shows one KITTI frame. The prediction captures the overall structure of the scene: the road gets gradually farther away towards the end of the street, the wall on the right is close, and the cyclists are visible.

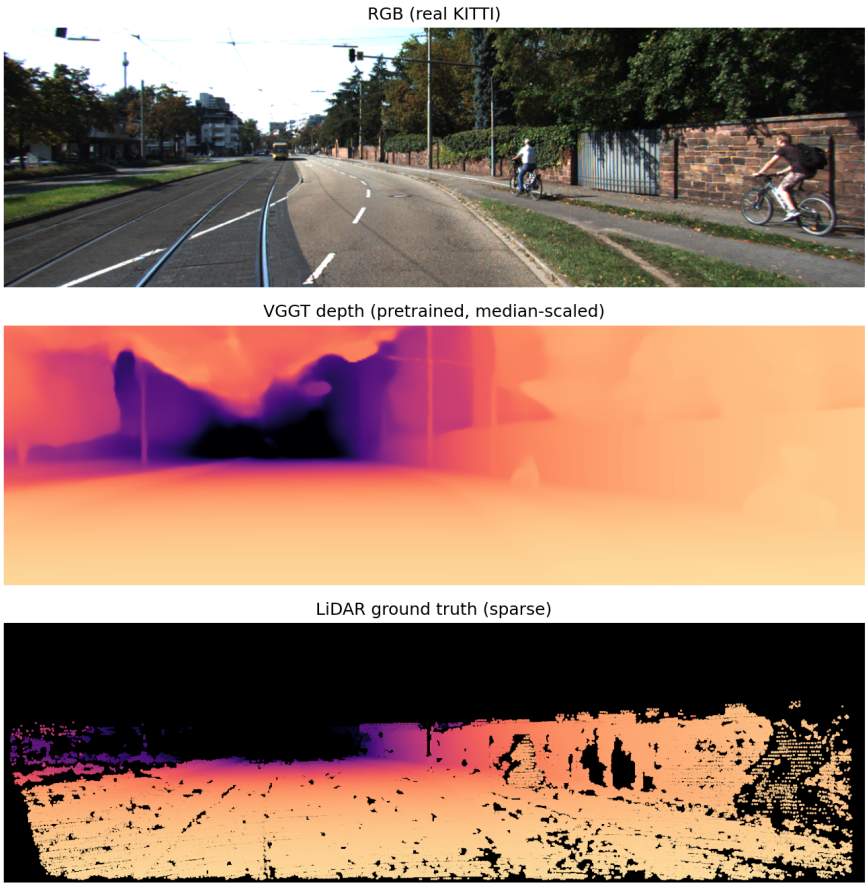


Figure 4: Pretrained VGGT on a real KITTI frame (drive `0002`). Top: input image. Middle: predicted depth after median scaling. Bottom: sparse LiDAR ground truth (black: no measurement). Lighter colours are closer.

7.6 Experiment 1: Full Resolution

In the first experiment, we used images 518 pixels wide, the same width VGGT was trained on, and followed the procedure of Section 7.3.

7.6.1 Training

Figure 5 shows the average training losses of each epoch. The total loss goes down over the whole training, and most of this decrease comes from the confidence term: the model learns where its predictions are reliable and becomes more confident there. The depth error term drops by about 33%, which shows that the depth head did adapt to the training scenes. The gradient term, which measures errors at object edges, changes much less. This suggests that the depth head mainly learns the overall depth values, while the fine details of the prediction depend mostly on the frozen aggregator.

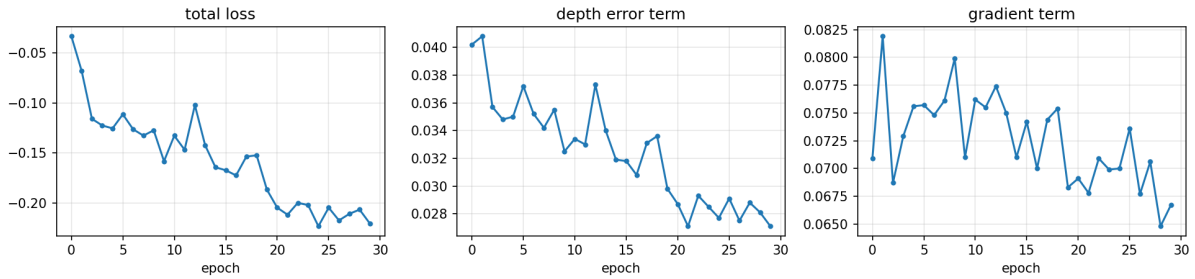


Figure 5: Average training losses per epoch on the four Virtual KITTI training scenes. The total loss is negative because of the $-\alpha \log \Sigma$ confidence term; lower is better.

7.6.2 Epoch Selection

Figure 6 shows the AbsRel of every epoch on the validation drives and, computed only after the choice, on the test drives. While the learning rate is high, the validation error goes up and down, and only about half of the epochs are better than the pretrained model. Once the learning rate becomes small, the curve settles at the pretrained level, and the last epoch ends there (0.0742 against 0.0739).

Epoch 5 has the lowest validation error, 0.0701, about 5% below the pretrained model. The next five epochs were all worse, so early stopping ended the search at epoch 10 and selected epoch 5. It is also the best of all 30 epochs, so early stopping did not miss a better one later.

The test curve has the same shape as the validation curve, with a correlation of 0.94. Epoch 5 is the best on the test drives too, and the epochs that jump up on the validation drives, such as 1, 6, and 10, jump up on the test drives as well. So the validation drives are a reliable guide for choosing an epoch.

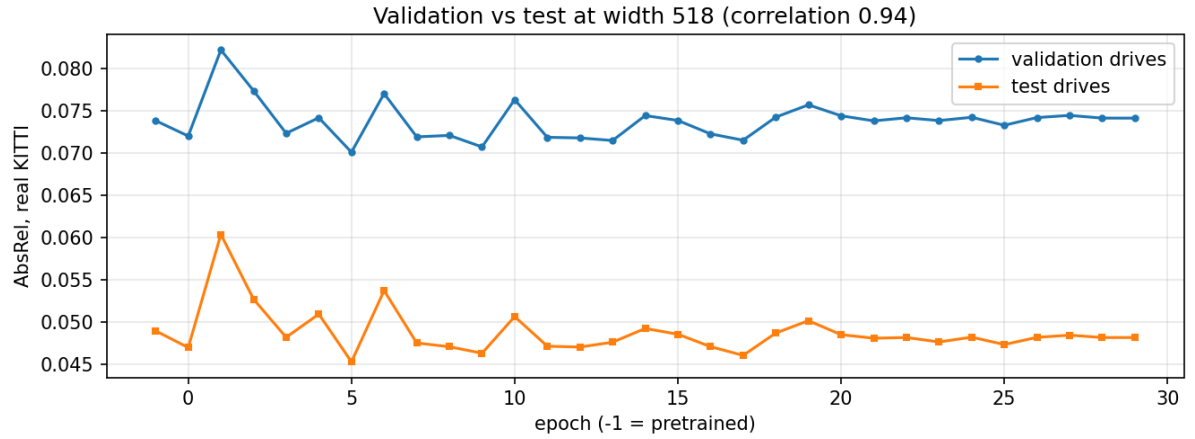


Figure 6: AbsRel of the pretrained model (epoch -1) and of every epoch on the real-KITTI validation and test drives. The choice uses only the validation drives; the test curve was computed afterwards, as a check.

7.6.3 Test Results

Table 4 compares the selected and the last depth head with the pretrained model, and Table 5 compares them frame by frame.

Test set / model	AbsRel ↓	SqRel ↓	RMSE ↓	RMSE log ↓	δ_1 ↑	δ_2 ↑	δ_3 ↑
Virtual KITTI – pretrained	0.0384	0.4814	4.145	0.1055	0.9743	0.9851	0.9924
Virtual KITTI – last (epoch 29)	**0.0370**	**0.4300**	**4.056**	**0.1039**	**0.9750**	**0.9855**	**0.9929**
Virtual KITTI – selected (epoch 5)	0.0404	0.4633	4.203	0.1100	0.9740	0.9843	0.9923
Real KITTI – pretrained	0.0489	0.2327	2.932	0.0861	0.9695	**0.9933**	**0.9974**
Real KITTI – last (epoch 29)	0.0481	0.2446	3.017	0.0870	0.9680	0.9925	0.9969
Real KITTI – selected (epoch 5)	**0.0453**	**0.2299**	**2.897**	**0.0841**	**0.9710**	0.9926	0.9968

Table 4: Results of Experiment 1 on both test sets (8-frame windows). "Last" is the depth head after the last epoch; "selected" is the one chosen with early stopping on the real-KITTI validation drives. Best value per test set in bold.

Model / test set	Frames	Mean AbsRel change	Frames improved
Selected – real KITTI	192	-0.0037 ± 0.0039	80%
Selected – Virtual KITTI	160	$+0.0020 \pm 0.0026$	18%
Last – real KITTI	192	-0.0008 ± 0.0030	72%
Last – Virtual KITTI	160	-0.0013 ± 0.0027	71%

Table 5: Per-frame comparison with the pretrained model: AbsRel of the fine-tuned model minus AbsRel of the pretrained model on the same frame (negative means that fine-tuning helped).

Selected model. On real KITTI, the selected model is better than the pretrained one on all main metrics, with AbsRel −7.4%. It is better on 80% of the frames, and on every frame of drives `0002` and `0013`. Drive `0005` is the exception: there, it is slightly worse on average and better on only 39% of the frames. On Virtual KITTI, the selected model is worse than the pretrained one (AbsRel +5.2%, better on only 18% of the frames).

Last epoch. The model after the last epoch shows the opposite picture. On Virtual KITTI, it is the best of the three models on every metric (AbsRel −3.6%, better on 71% of the frames). On real KITTI, its AbsRel is only slightly lower than the pretrained one (−1.6%), and SqRel, RMSE, RMSE log, and $\delta < 1.25$ are all worse. So it is better by a little on many frames, but worse by more on some, mostly on drive `0005`.

Interpretation. The depth head adapted to the *synthetic* look of the images: Virtual KITTI is rendered by a game engine, with clean textures, simple lighting, and no camera noise, while real photos differ in all of these ways. The selected epoch comes early in the training, while the learning rate is still high, at a point where the depth head suits real roads better but synthetic scenes worse. So the gain on real images does not come from learning the synthetic data better, but from a trade-off between the two domains, and without a check on real data we would not have found it.

7.6.4 View-Count Ablation

Next, we wanted to see how the number of views changes the depth quality. We evaluated the pretrained and the selected model on the same 192 real-KITTI test frames, with windows of 2, 4, 8, and 16 consecutive frames (Table 6).

Views	Pretrained	Selected (epoch 5)	Difference
2	0.0587	**0.0539**	−0.0048 (−8.2%)
4	0.0541	**0.0498**	−0.0043 (−8.0%)
8	0.0489	**0.0453**	−0.0037 (−7.5%)
16	0.0450	**0.0422**	−0.0029 (−6.3%)

Table 6: AbsRel on the real-KITTI test frames for different numbers of input views.

Two results are clear. First, **more views give better depth** for both models. Going from 2 to 16 views lowers the AbsRel of the pretrained model by 23%. This confirms, on real driving data the model never saw, the claim of Wang et al. (2025) that VGGT benefits from more views. Since the car moves forward, each extra frame shows the same road from a new position, and the global attention layers can use this to understand the 3D shape of the scene better.

Second, **the gain from fine-tuning stays clear for every number of views**. It gets only a little smaller as the number of views grows, from 8.2% with 2 views to 6.3% with 16. So the two improvements mostly add up: the selected model with 16 views has the lowest error of all (0.0422).

7.6.5 Further Tests

Finally, we tried three changes that could make the gain larger. We tested each one separately and made all choices on the validation drives, in the same way as before (Table 7).

Variant	Validation	Real KITTI	Virtual KITTI
Pretrained	0.0739	0.0489	0.0384
Main run, selected (epoch 5)	0.0701	0.0453	0.0404
Saving every 50 steps, selected	0.0695	0.0446	0.0542
Learning rate 10^{-5}, selected	0.0739	0.0489	0.0384
Learning rate 10^{-5}, last epoch	0.0776	0.0506	0.0368

Table 7: AbsRel of the further tests at width 518 (8-frame windows). With a learning rate of 10^{-5}, no epoch was better than the pretrained model on the validation drives, so the selection kept the pretrained depth head.

Averaging. We started from the selected epoch and added the next best epochs to the average one by one, keeping an epoch only if the validation error went down. No epoch passed this test: every average was slightly worse than epoch 5 alone (0.0705 to 0.0711, against 0.0701). So averaging gave no gain, and the result stayed epoch 5.

Saving more often. In the main run, we saved the depth head every 100 steps, so we could have missed a better one between two saves. To check this, we repeated the training and saved every 50 steps. The best checkpoint of this run was only slightly better than epoch 5 on the validation drives (0.0695 against 0.0701) and on real KITTI (0.0446 against 0.0453), and much worse on Virtual KITTI (Table 7). So saving more often did not give a clear gain.

Smaller learning rate. We lowered the learning rate from 5×10^{-5} to 10^{-5}, to check whether smaller updates reduce overfitting to the synthetic data. With this setting, no epoch was better than the pretrained model on the validation drives. The last epoch was the best of all models on Virtual KITTI, but worse than the pretrained model on real KITTI. So the depth head still adapted to the synthetic data, but it never passed through the states that suit real images better. These states seem to appear only with the larger updates of the higher learning rate.

None of the three changes gave a clear improvement, so we kept the main run.

7.7 Experiment 2: Lower Resolution

In the second experiment, we reduced the image width from 518 to 336 pixels, so that a KITTI frame becomes 336×98 pixels instead of 518×154. Each image then gives 168 tokens instead of 407, about 2.4 times fewer. Since the cost of attention grows with the square of the number of tokens, the attention layers become roughly six times cheaper. VGGT's own preprocessing always uses a width of 518, so we wrote our own resizing function and checked that, at a width of 518, it gives exactly the same images as VGGT's.

Apart from the resolution, we followed exactly the same procedure, with training, selection, and testing all at width 336. As the baseline, we evaluated the pretrained model at width 336 as well. Table 8 shows the results, next to the pretrained model at the normal width of 518 pixels.

Test set / model	AbsRel ↓	SqRel ↓	RMSE ↓	RMSE log ↓	δ_1 ↑	δ_2 ↑	δ_3 ↑
Real KITTI – pretrained, width 518	0.0489	0.2327	2.932	0.0861	0.9695	0.9933	0.9974
Real KITTI – pretrained, width 336	0.0897	0.5234	4.476	0.1394	0.9145	0.9813	0.9935
Real KITTI – last (epoch 29), width 336	0.0692	0.4103	3.912	0.1167	0.9425	0.9855	**0.9945**
Real KITTI – selected (epoch 14), width 336	**0.0675**	**0.3942**	**3.771**	**0.1130**	**0.9467**	**0.9861**	**0.9945**
Virtual KITTI – pretrained, width 518	0.0384	0.4814	4.145	0.1055	0.9743	0.9851	0.9924
Virtual KITTI – pretrained, width 336	0.0784	1.0022	6.022	0.1557	0.9480	0.9767	0.9859
Virtual KITTI – last (epoch 29), width 336	**0.0618**	**0.7316**	5.285	**0.1360**	0.9561	**0.9789**	0.9890
Virtual KITTI – selected (epoch 14), width 336	0.0623	0.7756	**5.168**	0.1373	**0.9575**	0.9786	**0.9899**

Table 8: Effect of the input resolution (8-frame windows). Bold: best of the three width-336 models on each test set.

Cost of the lower resolution. The lower resolution has a high price. For the pretrained model, the error almost doubles: AbsRel rises from 0.0489 to 0.0897 on real KITTI

(+83%) and from 0.0384 to 0.0784 on Virtual KITTI (+104%). This is not surprising, since the model has never seen this input size, and each token now has to describe a larger part of the scene.

Epoch selection. At this resolution, every one of the 30 epochs was better than the pretrained model on the validation drives (AbsRel between 0.0802 and 0.0937, against 0.1051). Epoch 14 had the lowest validation error, the next five epochs were all worse, so early stopping ended at epoch 19 and selected epoch 14, which is also the best of all 30 epochs. As in Experiment 1, the test curve has the same shape as the validation curve, with a correlation of 0.96 (Figure 7).

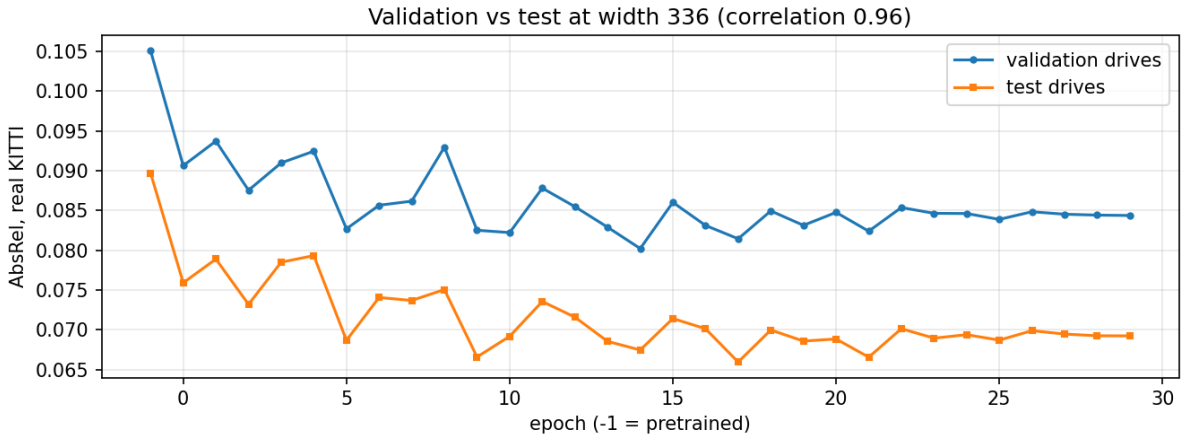


Figure 7: AbsRel of the pretrained model (epoch −1) and of every epoch at width 336, on the real-KITTI validation and test drives. The choice uses only the validation drives; the test curve was computed afterwards, as a check.

Effect of fine-tuning. Here we compare the fine-tuned models with the pretrained model at the same width of 336 pixels. Fine-tuning helped on both datasets. On real KITTI, the selected model lowered AbsRel from 0.0897 to 0.0675 (−25%), improved every metric, and was better on 96% of the frames. Unlike at width 518, it was also better on drive `0005` (on 89% of its frames). On Virtual KITTI, it lowered AbsRel by 21%. Lowering the resolution had raised the AbsRel on real KITTI from 0.0489 to 0.0897, and fine-tuning brought it back down to 0.0675, so it recovered about half of this loss. The model after the last epoch gave almost the same result (−23% on real KITTI), so at this width early stopping added only a little.

Interpretation. Unlike at width 518, here both models improve on both datasets. We think the reason is what the depth head has to learn in each case. At width 336, it mainly has to adapt to the new, smaller input size, and this adaptation is the same for synthetic and real images, so it carries over to real KITTI. At width 518, the input size is the one VGGT was trained for, so what the depth head mainly adapts to is the synthetic look of

Virtual KITTI, and this does not carry over. Even after fine-tuning, though, the model at width 336 is still clearly less accurate than the pretrained model at full resolution (0.0675 against 0.0489). Figure 8 shows the difference between the two widths.

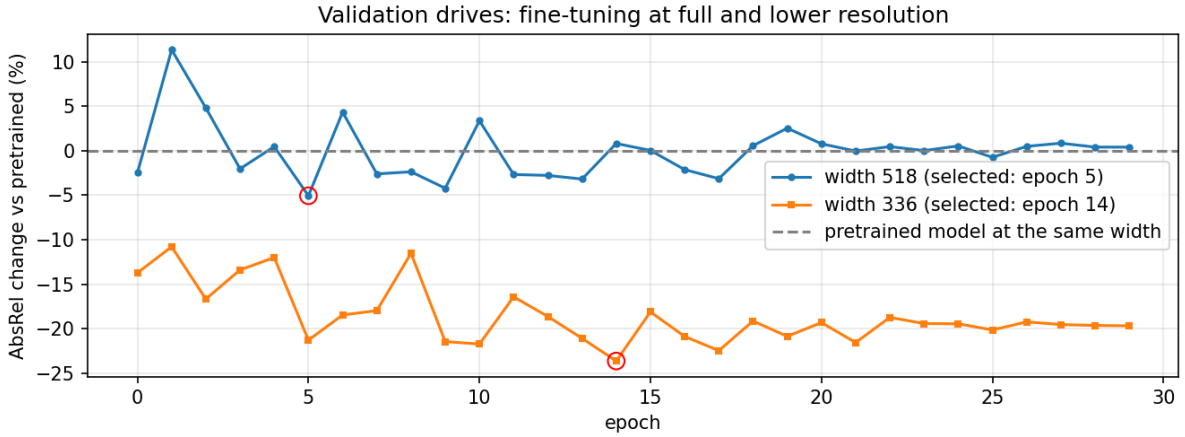


Figure 8: Change of the validation AbsRel of every epoch, relative to the pretrained model at the same width. Circles mark the selected epochs.

8 Conclusion

In this project, we fine-tuned only the depth head of VGGT on synthetic driving scenes from Virtual KITTI 2, with the aggregator frozen, and tested it on both Virtual KITTI and real KITTI images that the model had never seen. We can now answer the four questions of the introduction.

Does fine-tuning improve depth on synthetic driving scenes? A little. The pretrained model was already very accurate on Virtual KITTI (AbsRel 0.038), and after the full training of 3,000 steps, AbsRel was 3.6% lower.

Does this improvement also hold on real images? Not by itself. After the last epoch, the model was only slightly better on real KITTI in AbsRel, and worse in RMSE and $\delta < 1.25$, probably because the depth head had adapted to the synthetic look of the images. However, the accuracy on real images went up and down during training. With early stopping on separate real validation drives, we found a model that was better than the pretrained one on all main metrics of the real test set, with AbsRel 7.4% lower and better results on 80% of the frames. This model was worse on Virtual KITTI, so the gain came from a trade-off between the two domains, not from learning the synthetic data better. Averaging the best epochs, saving more often, or using a smaller learning rate did not make this gain clearly larger.

How does the number of views change the depth quality? A lot. Going from 2 to 16 views lowered the error of the pretrained model on real KITTI by 23%, which confirms that VGGT makes good use of several views, also on real driving scenes. The gain from fine-tuning stayed clear for every number of views, from 8.2% with 2 views to 6.3% with 16.

How does a lower resolution change the depth quality? Reducing the image width from 518 to 336 pixels made the attention layers about six times cheaper, but it almost doubled the error of the pretrained model. Here, fine-tuning the depth head clearly helped: it lowered the error on real KITTI by 25% and won back about half of the lost accuracy. Even so, the model at width 336 remained less accurate than the pretrained model at full resolution.

Overall, fine-tuning the depth head helped most when the model got less information than it was trained for: the gain on real KITTI was 25% at the lower resolution against 7.4% at full resolution, and it was slightly larger with fewer views.

Limitations. Our results have three limits. First, the test set is small, with only three drives and 192 frames, so the results should be confirmed on more data. Second, we trained each configuration only once, with one random seed, so we do not know how much the results would change in another run. Third, we rescaled every prediction to the median of the ground truth before computing the metrics. So the metrics show whether the depth is right relative to the rest of the scene, but not whether the distances in metres are right. Finally, since the main obstacle seems to be the gap between synthetic and real images, the most direct next step would be to fine-tune the depth head on real KITTI, or on a mix of real and synthetic data.

References

Cabon, Y., Murray, N., and Humenberger, M. (2020). Virtual KITTI 2. *arXiv preprint arXiv:2001.10773*.

Eigen, D., Puhrsch, C., and Fergus, R. (2014). Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Garg, R., B G, V. K., Carneiro, G., and Reid, I. (2016). Unsupervised CNN for single view depth estimation: Geometry to the rescue. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

Geiger, A., Lenz, P., Stiller, C., and Urtasun, R. (2013). Vision meets robotics: The KITTI dataset. *International Journal of Robotics Research (IJRR)*.

Hartley, R. and Zisserman, A. (2004). *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition.

Kendall, A. and Gal, Y. (2017). What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Leroy, V., Cabon, Y., and Revaud, J. (2024). Grounding image matching in 3D with MASt3R. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*.

Ranftl, R., Bochkovskiy, A., and Koltun, V. (2021). Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Schönberger, J. L. and Frahm, J.-M. (2016). Structure-from-motion revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Uhrig, J., Schneider, N., Schneider, L., Franke, U., Brox, T., and Geiger, A. (2017). Sparsity invariant CNNs. In *International Conference on 3D Vision (3DV)*.

Wang, J., Chen, M., Karaev, N., Vedaldi, A., Rupprecht, C., and Novotny, D. (2025). VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Wang, J., Karaev, N., Rupprecht, C., and Novotny, D. (2024a). VGGSfM: Visual geometry grounded deep structure from motion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Wang, S., Leroy, V., Cabon, Y., Chidlovskii, B., and Revaud, J. (2024b). DUSt3R: Geometric 3D vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Zhou, T., Brown, M., Snavely, N., and Lowe, D. G. (2017). Unsupervised learning of depth and ego-motion from video. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.